Übungsblatt 2: Scheduling-Theorie und kooperativer Scheduler

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Ziele, Voraussetzungen und Zeitplan

Sie unterscheiden Ausführungs- und Antwortzeit, prüfen Fristen unter verschiedenen Auswahlregeln und setzen einen kooperativen Dispatcher um. Vorausgesetzt werden **nur Vorlesungen 1 und 2**: Bare-Metal, Interrupts, C-Funktionen und das Aufgabenmodell. Ein Echtzeitbetriebssystem wird für den Programmierteil nicht eingesetzt. FreeRTOS und fortsetzbare Threads folgen in Vorlesung 3.

Teil A enthält fünf klausurnahe Papieraufgaben mit insgesamt 32 Punkten. Planen Sie etwa 50 Minuten einschließlich kurzer Auswertung. Teil B benötigt mit vorbereitetem Projekt etwa 45 Minuten; die zusätzliche Hardwaremessung etwa 15 Minuten. Papierlösungen sind auch ohne Hardware vollständig möglich. Die Installation der Werkzeuge und ein erstmaliger Dual-Core-Start sind Vorbereitung.

Teil B verwendet **STM32CubeMX und VS Code mit STM32 VS Code Extension** auf dem **NUCLEO-H755ZI-Q mit STM32H755ZIT6**. Die Anwendungsfunktionen stehen vollständig in der Musterlösung dieses Blatts. Ein generiertes Firmwareprojekt liegt im Repository nicht vor. Konfiguration, Build und Hardwarelauf werden deshalb am bereitgestellten Projekt nachgewiesen, nicht aus den Zahlenbeispielen behauptet.

Teil A: Papieraufgaben

Gemeinsames Modell und Notation

Sofern nicht anders angegeben: genau ein Prozessor, unabhängige periodische Aufgaben, vollständig präemptierbare Jobs, keine Sperren oder Selbstsuspendierung, kein Freigabejitter. Interrupt-, Auswahl- und Wechselkosten sind null. Ausführungszeiten sind vorgegebene obere Schranken. Diese **Lehrannahmen** sind keine Boardmessung und gelten nicht automatisch für den kooperativen Code in Teil B.

Größe	Bedeutung
C_i	eigener Prozessorbedarf eines Jobs der Aufgabe τ_i
T_i	Abstand periodischer Freigaben
D_i	relative Abschlussfrist ab Freigabe

Größe	Bedeutung
$r_{i,k}$, $s_{i,k}$, $f_{i,k}$	Freigabe, erster Start, Abschluss des Jobs k
$d_{i,k} = r_{i,k} + D_i$	absolute Deadline
$R_{i,k} = f_{i,k} - r_{i,k}$	Antwortzeit des einzelnen Jobs
R_i	nachgewiesene maximale Antwortzeit im Modell

Task bedeutet wiederkehrende Aufgabe, Job eine einzelne Aktivierung. Zeitintervalle sind halboffen: $[a, b)$ enthält den Anfang, aber nicht den Endpunkt. Abschluss genau auf der Deadline gilt als rechtzeitig.

Aufgabe 1: Ein Job auf der Zeitachse · 6 Punkte

Eine Aufgabe hat $T = 10\,\text{ms}$ und $D = 8\,\text{ms}$. Ein Job wird bei $r = 0$ freigegeben. Er wartet bis $s = 2$, rechnet von 2 bis 3, wird von einer anderen Aufgabe verdrängt und rechnet von 5 bis 7 weiter. Alle Zeitpunkte sind in Millisekunden.

1. Zeichnen Sie die Belegung dieses Jobs in $[0, 10)$. Kennzeichnen Sie Freigabe, Start, beide Rechenstücke, Abschluss, absolute Deadline und nächste Freigabe. (3 P)
2. Berechnen Sie eigene Prozessorzeit, Startlatenz und Antwortzeit. Prüfen Sie die Frist. Warum sind beide Rechenstücke derselbe Job? (2 P)
3. Ist die verdrängte Aufgabe von 3 bis 5 Ready oder Blocked? Begründen Sie anhand ihrer Warteursache. (1 P)

Musterlösung Aufgabe 1

Intervall / Zeitpunkt in ms	Bedeutung für diesen Job
0	Freigabe
$[0, 2)$	Ready: bereit, noch ohne Zuteilung
$[2, 3)$	Running: erstes Rechenstück
$[3, 5)$	Ready: durch andere Aufgabe verdrängt
$[5, 7)$	Running: Fortsetzung desselben Jobs
7	Abschluss
8	absolute Deadline
10	nächste periodische Freigabe

Eigene Prozessorzeit: $(3 - 2) + (7 - 5) = 3\,\text{ms}$. Startlatenz: $s - r = 2\,\text{ms}$. Antwortzeit: $R = f - r = 7\,\text{ms}$. Absolute Deadline: $d = r + D = 8\,\text{ms}$. Die Frist hält, weil $7 \leq 8$; das Restbudget beträgt 1 ms. Eine Unterbrechung erzeugt keinen neuen Job: Beide Laufstücke gehören zur Freigabe bei null.

Ready heißt ausführungsbereit, Blocked auf eine sachliche Bedingung oder Wartezeit wartend. Die Aufgabe könnte sofort weiterrechnen, wenn sie den Prozessor bekäme. Sie ist deshalb verdrängt und Ready, nicht Blocked. Der Abstand zwischen Freigabe und Abschluss enthält auch fremde Arbeit und ist nicht mit dem eigenen Bedarf C gleichzusetzen.

Aufgabe 2: RMS-Schranke richtig interpretieren · 6 Punkte

Für alle drei Aufgaben gilt $D_i = T_i$:

Aufgabe	C_i in ms	T_i in ms
τ_1	1	4
τ_2	2	6
τ_3	1	12

1. Berechnen Sie $U = \sum_i C_i/T_i$ als Bruch und Prozentwert. (2 P)
2. Vergeben Sie Prioritäten nach RMS, Rate Monotonic Scheduling. Prüfen Sie die hinreichende Liu-und-Layland-Schranke $U \leq n(2^{1/n} - 1)$ für $n = 3$. (2 P)
3. Was würde ein Überschreiten dieser Schranke bei weiterhin $U \leq 1$ bedeuten? Darf der bestandene Test unverändert auf den kooperativen Round-Robin aus Teil B übertragen werden? Begründen Sie beide Antworten. (2 P)

Musterlösung Aufgabe 2

$$U = \frac{1}{4} + \frac{2}{6} + \frac{1}{12} = \frac{3}{12} + \frac{4}{12} + \frac{1}{12} = \frac{2}{3} \approx 66{,}67\,\%.$$

RMS vergibt die höchste feste Priorität an die kürzeste Periode: $\tau_1 > \tau_2 > \tau_3$. Für drei Aufgaben beträgt die Schranke $3(2^{1/3} - 1) \approx 0{,}779763$, also etwa 77,98 %. Unser Wert liegt darunter; das Set ist unter den gemeinsamen Modellannahmen mit RMS nachgewiesen.

Die Schranke ist hinreichend, nicht notwendig: Bei Überschreitung bis 100 % bleibt die Entscheidung offen. Dann kann eine genauere Antwortzeitanalyse den konkreten Satz prüfen. Dauerhafte Pflichtlast mit $U > 1$ passt dagegen nicht auf einen Prozessor.

Der Nachweis gilt nicht für beliebige Auswahlregeln. Der Labor-Dispatcher verwendet weder RMS-Prioritäten noch Task-Präemption. Lange zurückkehrende Schritte können andere Aufgaben verzögern. Gleiche Namen oder dieselbe Auslastung ersetzen diese fehlenden Voraussetzungen nicht.

Aufgabe 3: Antwortzeit trotz überschrittener Schranke · 8 Punkte

Dieses Set entspricht dem 85-%-Beispiel der Vorlesung; es ist **ein anderes Set als in Aufgabe 2**. Wieder gilt $D_i = T_i$:

Aufgabe	C_i in ms	T_i in ms
τ_1	1	4
τ_2	2	5
τ_3	2	10

Die RTA, Response-Time Analysis oder Antwortzeitanalyse, verwendet für feste Prioritäten:

$$R_i^{(0)} = C_i, \qquad R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j.$$

$hp(i)$ ist die Menge höher priorisierter Aufgaben, $\lceil x \rceil$ rundet auf die nächste ganze Zahl auf. Iterieren Sie bis zu einem unveränderten Wert oder einer Überschreitung der Deadline.

1. Bestimmen Sie U und die RMS-Reihenfolge. Welche Aussage liefert allein die Liu-und-Layland-Schranke? (2 P)
2. Berechnen Sie R_1, R_2 und alle Iterationswerte für R_3 mit Zwischenrechnungen. (4 P)
3. Prüfen Sie jede Frist und erklären Sie, warum $\lceil 5/5 \rceil = 1$ gilt. Darf aus dem Ergebnis „85 % ist unter RMS immer sicher“ folgen? (2 P)

Musterlösung Aufgabe 3

$U = 1/4 + 2/5 + 2/10 = 0{,}85 = 85\,\%$, Rangfolge $\tau_1 > \tau_2 > \tau_3$. Die einfache Schranke von etwa 77,98 % ist überschritten und liefert keine Entscheidung.

τ_1 hat keinen höher priorisierten Störer: $R_1 = C_1 = 1\,\text{ms} \le 4\,\text{ms}$.

Für τ_2: Startwert 2, danach $2 + \lceil 2/4 \rceil \cdot 1 = 3$, nochmals $2 + \lceil 3/4 \rceil \cdot 1 = 3$. Damit $R_2 = 3\,\text{ms} \le 5\,\text{ms}$.

Iteration für τ_3	Rechnung	Ergebnis in ms
0	C_3	2
1	$2 + \lceil 2/4 \rceil \cdot 1 + \lceil 2/5 \rceil \cdot 2$	5
2	$2 + \lceil 5/4 \rceil \cdot 1 + \lceil 5/5 \rceil \cdot 2$	6
3	$2 + \lceil 6/4 \rceil \cdot 1 + \lceil 6/5 \rceil \cdot 2$	8
4	$2 + \lceil 8/4 \rceil \cdot 1 + \lceil 8/5 \rceil \cdot 2$	8

Fixpunkt $R_3 = 8\,\text{ms} \le 10\,\text{ms}$. Alle Fristen halten im angegebenen RMS-Modell. Bei einem Fenster der Länge fünf ist ein Job mit Periode fünf relevant; die nächste Freigabe liegt genau am rechten, ausgeschlossenen Rand. Ein bereits dort abgeschlossener betrachteter Job wird von ihr nicht rückwirkend verzögert. Darum wird in diesem Iterationsschritt nicht auf zwei erhöht. Sobald das berechnete Fenster auf sechs Millisekunden wächst, liegt die Freigabe bei fünf im Inneren; die nächste Iteration zählt daher zwei Jobs von dieser Aufgabe.

Die konkreten Perioden und Bedarfe tragen den Nachweis. Ein anderes Set mit 85 % kann eine andere Antwortzeit haben. Prozessorzeit und Antwortzeit sind auch hier verschieden: $C_3 = 2$, aber $R_3 = 8$ ms.

Aufgabe 4: RMS und EDF am selben Set · 6 Punkte

Zwei Aufgaben starten synchron bei null:

Aufgabe	C_i in ms	$T_i = D_i$ in ms
τ_1	2	5
τ_2	4	7

EDF bedeutet Earliest Deadline First: Aus den bereiten Jobs wird der mit der frühesten **absoluten** Deadline gewählt. Bei Gleichstand bleibt der laufende Job. Es gilt das gemeinsame präemptive Modell.

1. Berechnen Sie U. Zeichnen Sie unter RMS die Belegung bis 7 ms und zeigen Sie die Fristverletzung. (2 P)
2. Zeichnen Sie unter EDF die Belegung bis 12 ms. Beschriften Sie die Jobs und begründen Sie die Entscheidungen bei 5 und 10 ms mit den absoluten Deadlines. (3 P)

3. Was bedeutet dieses Ergebnis für den Unterschied zwischen „unter RMS nicht einplanbar“ und „überhaupt nicht machbar“? Begründen Sie die EDF-Aussage für alle periodischen Jobs, nicht nur den gezeichneten Ausschnitt. (1 P)

Musterlösung Aufgabe 4

$U = 2/5 + 4/7 = 34/35 \approx 97{,}14\,\%$. RMS wählt τ_1 vor τ_2:

RMS-Intervall in ms	Laufender Job
$[0, 2)$	τ_1/J_1
$[2, 5)$	τ_2/J_1, nur 3 von 4 ms erreicht
$[5, 7)$	τ_1/J_2

Bei 7 ms fehlen dem ersten Job von τ_2 noch 1 ms; seine absolute Deadline sieben ist verletzt. Alternativ ergibt seine RTA $4 \to 6 \to 8 > 7$.

EDF-Intervall in ms	Laufender Job	Absolute Deadline in ms
$[0, 2)$	τ_1/J_1	5
$[2, 6)$	τ_2/J_1	7
$[6, 8)$	τ_1/J_2	10
$[8, 12)$	τ_2/J_2	14

Bei 5 ms bleibt τ_2/J_1 laufend: Deadline sieben ist früher als die Deadline zehn des neu freigegebenen τ_1/J_2. Bei 10 ms hat der laufende τ_2/J_2 Deadline vierzehn, der neue τ_1/J_3 Deadline fünfzehn. Daher bleibt auch dort der laufende Job. Bei 12 ms ist er fertig und τ_1/J_3 kann bis 14 ms rechnen.

Das Scheitern einer Auswahlregel beweist keine Unmöglichkeit jeder Auswahlregel. Für unabhängige vollständig präemptierbare periodische Aufgaben mit $D_i = T_i$ und ohne Zusatzlast ist $U \leq 1$ der EDF-Nachweis. Der Test belegt im benannten Idealmodell alle periodischen Jobs; die kurze Zeichnung illustriert ihn nur. Bei kürzeren Deadlines oder zusätzlichen Kosten genügt diese Aussage nicht unverändert.

Aufgabe 5: Kooperative Runden und Jitter · 6 Punkte

Hier gilt ausdrücklich ein **anderes Modell**: Drei Schritte A, B und C sind ständig bereit. Der Dispatcher ruft sie zyklisch A, B, C auf. Jeder Schritt kehrt nach seiner Arbeit zurück. Es gibt keine erzwungenen Zeitscheiben und keine periodischen Jobfreigaben. Erster A-Start ist bei null; Aufruf- und Interruptkosten sind null.

A benötigt 1 ms, C ebenfalls 1 ms. Ein Marker ist während der Arbeit von A High. Definieren Sie Pulsbreite $w_k = f_k - s_k$, Startabstand $P_k = s_{k+1} - s_k$ und Peak-to-Peak-Jitter $J_P = \max(P_k) - \min(P_k)$.

1. Bestimmen Sie die ersten fünf A-Starts, Pulsbreite, Startabstände und J_P bei konstant 1 ms für B. Wiederholen Sie für konstant 3 ms für B. Ist die konstante Verlängerung allein schon Jitter? (2 P)
2. B benötigt abwechselnd 1, 3, 1, 3 ms. Bestimmen Sie die ersten fünf A-Starts, die vier Startabstände, Pulsbreite und J_P. (2 P)

3. B kehrt stattdessen nie zurück. Was geschieht mit weiteren A-/C-Aufrufen und mit weiterhin erlaubten Tickinterrupts? Warum ist `wake_at` setzen ohne Rückkehr kein Taskwechsel, und warum liefert eine Wake-Zeit keinen genauen Startzeitpunkt? (2 P)

Musterlösung Aufgabe 5

B-Dauer	Erste fünf A-Starts in ms	Startabstände in ms	Pulsbreite	J_P
konstant 1 ms	0, 3, 6, 9, 12	3, 3, 3, 3	1 ms	0 ms
konstant 3 ms	0, 5, 10, 15, 20	5, 5, 5, 5	1 ms	0 ms
abwechselnd 1/3 ms	0, 3, 8, 11, 16	3, 5, 3, 5	1 ms	2 ms

Jede Runde braucht $C_A + C_B + C_C$. Konstante Mehrarbeit verlängert die Abstände konstant; erst ihre Schwankung erzeugt hier Startabstandsjitter. Die Pulsbreite von A bleibt im Modell eine Millisekunde, weil A selbst unverändert arbeitet und keine Unterbrechung angenommen ist. Auf dem Board können Interrupts die verstrichene Markerbreite verändern.

Ohne Rückkehr aus B bleibt der Dispatcher in B. Weitere A- und C-Aufrufe bleiben aus, obwohl zulässige Interrupts die Zeitbasis weiterhin bedienen können. Eine Hilfsfunktion, die Zustand und `wake_at` ändert, beendet nicht den aufrufenden Schritt. Dieser muss selbst `return` ausführen. Nach erreichtem Wake-Ziel wird ein Schritt nur Ready; der nächste Aufruf hängt von den noch laufenden Schritten und der Ringauswahl ab. Der Laborcode besitzt keinen eigenen pausierten Stack je Schritt.

Teil B: Kooperativen Dispatcher auf dem Board untersuchen

Aufgabe 6: Projekt vorbereiten und eigene Funktionen erstellen

Verwenden Sie eine funktionierende Dual-Core-Bare-Metal-Basis für das **NUCLEO-H755ZI-Q** aus Vorlesung 1. CM4 bezeichnet den Cortex-M4-Projektteil, CM7 den Cortex-M7-Projektteil. Die heutige Anwendung läuft nur auf CM4; der vorbereitete CM7-Boot- und Systemstart bleibt erhalten. Der zweite Kern verdoppelt den Ein-Prozessor-Nachweis aus Teil A nicht.

1. Öffnen Sie die Basis in **STM32CubeMX**. Prüfen Sie Board, STM32H755ZIT6, CM4-/CM7-Ziele und die für **VS Code mit STM32 VS Code Extension** benötigte CMake-Generierung. CMake beschreibt Übersetzung und Verknüpfung. Nutzen Sie die Versionen und Importmöglichkeiten der installierten Erweiterung. Build, Flash und Debug erfolgen über die genannte VS-Code-Erweiterung.
2. Aktivieren Sie **kein FreeRTOS**. Prüfen Sie die laufende HAL-Zeitbasis auf CM4: `HAL_GetTick()` muss die 32-Bit-Zeit in Millisekunden fortschalten, üblicherweise mit 1-kHz-Interrupt. HAL bedeutet Hardware Abstraction Layer, Hardwareabstraktion. Übernehmen Sie eine tatsächlich funktionierende Zeitbasis, nicht eine unbestätigte Taktannahme. Der Tickinterrupt wählt keinen anderen Schritt aus.
3. Deaktivieren Sie zusätzliche Anwendungslasten und den Timer-LED-Versuch aus Übung 1. Der CM7 darf nicht parallel denselben Markerpin schreiben. Interrupts für die Zeitbasis bleiben aktiv.
4. Wählen Sie anhand von Boardrevision, Schaltplan und Belegung einen freien, herausgeführten GPIO-Pin. GPIO bedeutet General Purpose Input/Output, allgemeiner Ein- und Ausgang. Geben Sie ihm in CubeMX das **User Label** `LAB_MARKER`, Ausgangsmodus und Anfangspegel Low. Ordnen Sie ihn dem CM4-Projekt zu und prüfen Sie Takt und

Initialisierung. CubeMX erzeugt dafür `LAB_MARKER_GPIO_Port` und `LAB_MARKER_Pin` in der zugehörigen `main.h`. PA0 ist nur eine Möglichkeit nach Prüfung der tatsächlichen Belegung, keine pauschale Vorgabe.

5. Generieren Sie die Quellen und öffnen Sie das erzeugte Projekt in VS Code. Prüfen Sie die CM4-/CM7-Buildziele, Flash- und Debugkonfiguration. Die erzeugte .ioc-Datei dokumentiert die CubeMX-Konfiguration. Ein erfolgreicher Build wird erst nach tatsächlichem Ausführen so eingetragen.

Implementieren Sie im vorgesehenen Benutzerbereich der **CM4-`main.c`**:

- Einen TCB, Task Control Block oder Verwaltungsdatensatz, mit Funktionszeiger `step`, Zustand Ready/Blocked und Wake-Zeit `wake_at`. Es gibt keinen Stackpointer pro Schritt.
- Drei endliche Schritte, die eine begrenzte Rechnung ausführen. Jeder schreibt seinen eigenen Ergebniswert. A setzt seinen Marker vor der Rechnung High und danach Low. B kann zwischen kurzer und längerer Rechnung wechseln. C arbeitet kurz.
- Eine zentrale Funktion `LabDispatchOnce()`: zuerst fällige Wake-Zeiten bearbeiten, danach höchstens drei Einträge ab dem Ringcursor prüfen, genau einen bereiten Schritt aufrufen und nach dessen Rückkehr zurückkehren. Niemand bereit: auf den weiter aktiven Tickinterrupt warten.
- Einen überlaufsicheren Vergleich der 32-Bit-Tickzeit. Alle relevanten Warteintervalle und Verspätungen bleiben für diese Übung strikt kleiner als 2^{31} Millisekunden; ein einfacher Vergleich `now >= wake_at` genügt beim Überlauf nicht.
- `LabSleepAfterStep(ms)`: Wake-Ziel und Blocked setzen. **Der aufrufende Schritt kehrt danach selbst sofort zurück.** Der Dispatcher wird nicht rekursiv aufgerufen.

Für die Zählerschleifen werden im Referenzcode 30000 und 120000 Iterationen verwendet. Das sind **keine zugesicherten Millisekundenzeiten**. Prüfen Sie Optimierung, Takte und beobachtbare Dauer. Falls nötig, passt die Lehrperson die beiden Zahlen vor dem Kurs an; alle Vergleichsläufe behalten danach denselben Build- und Taktstand.

Aufgabe 7: Drei Varianten vergleichen

1. **Kurze konstante Schritte:** Alle drei Schritte bleiben Ready. Kontrollieren Sie die fortschreitenden Aufrufzähler `lab_runs` und Ergebnisse `lab_result`. Der Marker gehört allein A. Eine Ringrunde erzwingt keine gleichen Zeitquanten.
2. **Lange konstante B-Arbeit:** Ändern Sie nur B auf die größere Iterationszahl. Startabstände von A werden voraussichtlich größer. Ohne Messung werden weder Abstände noch Jitterwerte behauptet.
3. **Wechselnde B-Arbeit:** B verwendet abwechselnd kleine und große Iterationszahl. Vergleichen Sie mit der konstanten langen Variante. Begründen Sie, welche Größe durch die Nachbararbeit verändert wird und welche zusätzlich von Interrupts abhängen kann.
4. **Kooperatives Schlafen von A:** Setzen Sie im Referenzcode `LAB_A_SLEEP_MS` auf 10, bauen und resetten Sie erneut. A setzt nach seiner Arbeit die Wake-Zeit auf aktuell plus zehn Millisekunden und kehrt zurück. B und C bleiben bereit. Der nächste A-Aufruf startet erst nach erreichtem Ziel und passender Ringauswahl. Das ist keine feste Releasefolge bei 0, 10, 20 ms. Die Referenz benutzt relatives Schlafen nach Arbeit, keinen periodischen RMS-Plan.
5. Lesen Sie die Dispatcherfunktionen bei angehaltenem Debugger. Welche Felder werden nur in der Hauptschleife geschrieben? Was bedeutet der ungültige Index in `running_idx` außerhalb eines Schrittaufrufs? Warum ist die Kandidatensuche begrenzt? Was würde bei drei Blocked-Einträgen passieren?

Keine verpflichtende Endlosschleifenprobe: Diagnostizieren Sie den fehlenden Rücksprung von B auf Papier. Zum Experiment genügt die begrenzte längere Last. Während eines Debuggerhalts wird das ursprüngliche Zeitverhalten verändert; Diagnosefelder sind keine ungestörte Laufzeitmessung.

Musterlösung Aufgaben 6–7: vollständiger Anwendungscode

Die folgende C-Referenz gehört in den vorgesehenen Benutzer-Funktionsbereich der CM4-`main.c`. Deren vorhandene `#include "main.h"` stellt die HAL- und Cortex-M-Funktionen sowie die generierten Markernamen bereit. Ergänzen Sie `<stdint.h>` im erhaltenen Include-Bereich. Der Code ist vollständig für die Anwendungslogik; Boot, Takte, GPIO und Interruptvektoren bleiben die Verantwortung der erzeugten Basis.

```c
#define LAB_N 3u
#define LAB_NONE UINT8_MAX
#define LAB_HALF_RANGE UINT32_C(0x80000000)
#define LAB_SMALL_WORK UINT32_C(30000)
#define LAB_LARGE_WORK UINT32_C(120000)
#define LAB_B_LONG 0u
#define LAB_B_VARIABLE 0u
#define LAB_A_SLEEP_MS 0u

typedef enum { ST_READY, ST_BLOCKED } TaskState;
typedef void (*TaskStep_t)(void);

typedef struct {
    TaskStep_t step;
    TaskState state;
    uint32_t wake_at;
} MyTCB_t;

static void TaskA_step(void);
static void TaskB_step(void);
static void TaskC_step(void);

static MyTCB_t task_list[LAB_N] = {
    { TaskA_step, ST_READY, 0u },
    { TaskB_step, ST_READY, 0u },
    { TaskC_step, ST_READY, 0u }
};
static uint8_t rr_cursor;
static uint8_t running_idx = LAB_NONE;

/* Debuggerdiagnose: nur die Hauptschleife schreibt diese Felder.
   volatile bietet keine allgemeine Synchronisation oder Zeitmessung. */
volatile uint32_t lab_runs[LAB_N];
volatile uint32_t lab_result[LAB_N];
volatile uint32_t lab_idle_entries;

static void LabFail(void)
{
    Error_Handler();
```

```
    for (;;) {}
}

static int LabTimeReached(uint32_t now, uint32_t target)
{
    /* Modulovergleich; alle relevanten Abstaende < 2^31 ms. */
    return (uint32_t)(now - target) < LAB_HALF_RANGE;
}

static void LabSleepAfterStep(uint32_t ms)
{
    if (running_idx >= LAB_N || ms == 0u || ms >= LAB_HALF_RANGE) {
        LabFail();
    }
    task_list[running_idx].wake_at = HAL_GetTick() + ms;
    task_list[running_idx].state = ST_BLOCKED;
    /* Aufrufender Schritt muss danach selbst sofort return ausfuehren. */
}

static void LabCpuWork(uint32_t iterations, uint8_t index)
{
    while (iterations-- > 0u) {
        lab_result[index] = lab_result[index] * UINT32_C(1664525)
                          + UINT32_C(1013904223);
    }
}

static void TaskA_step(void)
{
    HAL_GPIO_WritePin(LAB_MARKER_GPIO_Port, LAB_MARKER_Pin, GPIO_PIN_SET);
    LabCpuWork(LAB_SMALL_WORK, 0u);
    HAL_GPIO_WritePin(LAB_MARKER_GPIO_Port, LAB_MARKER_Pin, GPIO_PIN_RESET);
    ++lab_runs[0];
    if (LAB_A_SLEEP_MS != 0u) {
        LabSleepAfterStep(LAB_A_SLEEP_MS);
        return;
    }
}

static void TaskB_step(void)
{
    uint32_t work = LAB_B_LONG ? LAB_LARGE_WORK : LAB_SMALL_WORK;
    if (LAB_B_VARIABLE != 0u) {
        work = ((lab_runs[1] & 1u) == 0u) ? LAB_SMALL_WORK : LAB_LARGE_WORK;
    }
    LabCpuWork(work, 1u);
    ++lab_runs[1];
}

static void TaskC_step(void)
{
```

```
    LabCpuWork(LAB_SMALL_WORK, 2u);
    ++lab_runs[2];
}

static void LabDispatchOnce(void)
{
    if (running_idx != LAB_NONE) {
        LabFail(); /* Kein rekursiver Dispatch aus einem Schritt. */
    }
    uint32_t now = HAL_GetTick();
    for (uint8_t i = 0u; i < LAB_N; ++i) {
        if (task_list[i].state == ST_BLOCKED &&
            LabTimeReached(now, task_list[i].wake_at)) {
            task_list[i].state = ST_READY;
        }
    }
    int selected = -1;
    for (uint8_t k = 0u; k < LAB_N; ++k) {
        uint8_t i = (uint8_t)((rr_cursor + k) % LAB_N);
        if (task_list[i].state == ST_READY) {
            selected = (int)i;
            break;
        }
    }
    if (selected < 0) {
        ++lab_idle_entries;
        __WFI(); /* Wait For Interrupt: aktive Wake-Quelle erforderlich. */
        return;
    }
    rr_cursor = (uint8_t)(((unsigned)selected + 1u) % LAB_N);
    running_idx = (uint8_t)selected;
    task_list[selected].step();
    running_idx = LAB_NONE;
}
```

Rufen Sie **innerhalb der vorhandenen generierten Hauptschleife** auf, nach erfolgreicher Boot-, HAL- und GPIO-Initialisierung:

```
/* USER CODE BEGIN 3: innerhalb der bereits vorhandenen while (1) */
LabDispatchOnce();
/* USER CODE END 3 */
```

Keine zweite `main`-Funktion, keine verschachtelte Dispatcher-Endlosschleife und kein Step-Aufruf aus einem anderen Step. Entfernen Sie die alte Anwendungsarbeit aus derselben Schleife. Falls der Funktionsbereich hinter `main` liegt, deklarieren Sie `static void LabDispatchOnce(void);` einmal im vorgesehenen Prototypbereich. Prüfen Sie einen Build und eine erneute CubeMX-Generierung auf Erhalt der Benutzerbereiche.

Varianten: Grundlauf mit allen drei Schaltern null; konstant lange B-Arbeit mit `LAB_B_LONG = 1`, `LAB_B_VARIABLE = 0`; wechselnde B-Arbeit mit `LAB_B_VARIABLE = 1`. In letzterem Fall entscheidet dieser Schalter über die kleine/große Folge. Für den Schlafversuch wird

`LAB_A_SLEEP_MS = 10` gesetzt, die B-Schalter werden zunächst wieder null. Reset vor jedem Lauf.

Erklärung: Der Ringcursor zeigt nach einer Auswahl auf den Nachfolger. Blockierte Einträge werden übersprungen; die Suche betrachtet höchstens drei Kandidaten. Wenn alle Schritte Blocked sind, wartet `__WFI`, Wait For Interrupt, auf eine aktive Interruptquelle. Die Tickbasis muss weiterlaufen; ein Tick, der schon vor dem Warten behandelt wurde, kann die Kontrolle erst beim folgenden Tick erneut zurückbringen. Dieses einfache Idle-Muster beansprucht keinen sofortigen Start am Wake-Ziel.

`running_idx` gilt nur zwischen Aufruf und Rückkehr eines Schritts. Außerhalb steht dort `LAB_NONE`. Das Speichern einer Wake-Zeit verändert die Liste, nicht den C-Aufrufstack. Die übrigen lokalen Berechnungen werden nicht an beliebigen Stellen fortgesetzt. Bei längerfristiger Fortsetzung muss der Schritt seinen Arbeitszustand ausdrücklich in Anwendungsdaten speichern.

Der Modulovergleich verwendet vorzeichenlose 32-Bit-Arithmetik und funktioniert auch beim Überlauf. Der Abstand muss strikt kleiner als die halbe Spanne sein; auch ein überfälliger Eintrag darf nicht länger als diese Spanne unbeobachtet bleiben. `wake_at = 0` ist ein gültiger Zeitpunkt, deshalb kennzeichnet das separate Zustandsfeld das Warten. Interrupts schreiben hier nur die von HAL verwaltete Zeitbasis; die TCB-Liste und Diagnosezähler haben genau einen Schreiber, den Dispatcher beziehungsweise den aufgerufenen Schritt. Die Referenz ist ein kooperativer Round-Robin, keine RMS-/EDF-Implementierung und kein präemptives Betriebssystem.

Zusatzaufgabe: Marker messen und Grenzen dokumentieren

Ein Oszilloskop oder Logikanalysator ist nur für diese Zusatzaufgabe erforderlich. Messen Sie den in CubeMX gewählten A-Marker; verbinden Sie Gerätemasse und Boardmasse und beachten Sie die zulässigen Pegel. Der Marker umfasst die A-Rechnung sowie Teile der GPIO-Aufrufe und mögliche Interruptunterbrechungen. Er markiert nicht die komplette Dispatcherfunktion.

- Erfassen Sie für jede Lastvariante mindestens 20 Pulse unter gleichem Takt und gleicher Optimierung. Dokumentieren Sie Pin/Boardanschluss, Tickquelle, Werkzeugversionen, Geräteauflösung, Samplezahl und Lastschalter.
- Bestimmen Sie $w_k = f_k - s_k$, $P_k = s_{k+1} - s_k$ sowie $J_P = \max(P_k) - \min(P_k)$ für Ihre endliche Messreihe. Trennen Sie Pulsbreite und Startabstand. Die Modellwerte aus Aufgabe 5 gehören nicht in die Messspalten.
- Vergleichen Sie konstante kurze, konstante lange und wechselnde B-Arbeit. Auch im konstanten Fall kann die reale Messung Schwankungen durch Interrupts zeigen.
- Erklären Sie, warum die größte gemessene Markerbreite keine allgemein nachgewiesene WCET, Worst-Case Execution Time oder größte eigene Ausführungszeit, ist.

Ohne Messgerät bleiben alle Hardwarezeiten leer. Dann werden Konfiguration, Buildstatus, Diagnosefelder und die begründete Ablaufprognose abgegeben. Sichtbares Blinken allein wäre ebenfalls kein Fristnachweis.

Musterlösung der Messauswertung

Konstante Mehrarbeit in B verlängert erwartbar den mittleren Abstand zwischen A-Starts. Wechselnde B-Arbeit kann diesen Abstand zusätzlich schwanken lassen. Die A-Rechnung selbst bleibt gleich; trotzdem kann die verstrichene Pulsbreite wegen Interrupts und Markeraufwand variieren. Die Zählschleifen sind ohne Messung keine bekannten Millisekundenbudgets.

Ein endlicher beobachteter Höchstwert erfasst weder jeden Programmpfad noch alle Störungen, und die Markerbreite enthält möglicherweise fremde Interruptarbeit. Ein allgemeiner WCET-Nachweis braucht ein dazu passendes Modell und begründete Grenzen. Relatives Schlafen nach A-Arbeit fügt mindestens die konfigurierte Tickdifferenz bis zur erneuten Bereitschaft hinzu; Start und Freigabe bleiben verschieden. Insbesondere beweist dieser Versuch weder die Liu-und-Layland-Annahmen noch die Deadline-Einhaltung der Sets aus Teil A.

Abgabe und Quellen

Teil A: Zeitachsen beziehungsweise Intervalltabellen, Rechnungen und begründete Entscheidungen zu Aufgaben 1–5, insgesamt 32 Punkte. Teil B: .ioc-Konfiguration, eigene Quellen, Werkzeug- und Buildangaben, Vergleich der Varianten und Erklärung der Dispatcherzustände. Kennzeichnen Sie jeweils **durchgeführt**, **fehlgeschlagen** oder **nicht durchgeführt**. Geben Sie Messergebnisse nur mit tatsächlichem Messprotokoll an.

Quellen: Vorlesungen 1–2; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*, 3. Auflage; Liu und Layland (1973) zur RMS-Schranke; die in Vorlesung 2 behandelte Antwortzeitanalyse; ST UM2408 und RM0399 für Board und Mikrocontroller sowie die Dokumentation der installierten STM32 VS Code Extension. Konkrete Pins, Startpfade und Zeitbasen werden an Ihrer bereitgestellten Projektversion geprüft.